

IT UNIVERSITY OF COPENHAGEN

Department of Computer Science
KIREPRO1PE - Master Research Project

Concurrent Hash Table Performance Across ARM Hardware Tiers

Prepared by: Vaiva Staugaityte (vais@itu.dk), Elias
Illeris Poggi (elpo@itu.dk)

Supervised by: Peter Sestoft

Date: May 15, 2026

Abstract

Concurrent hash table implementations are most often benchmarked on high-end x86 hardware with large last-level caches and many cores, leaving their behavior on resource-constrained hardware poorly understood. We evaluate eight Java concurrent hash table implementations across two ARM hardware tiers: a Raspberry Pi 5 and an NVIDIA DGX Spark HPC node. Using the Java Microbenchmarking Harness, every thread count is tested using 144 configurations with a varying read ratio, key range, and access distribution across varying hash table implementations. We find that the performance hierarchy established in prior benchmarking on x86 systems, in which `WrapConcurrentHashMap` has the highest average throughput and `SynchronizedMap` performs worst, transfers to both ARM platforms. The most significant hardware-sensitive result is from the `HashTrie` implementation, which ranks seventh on the RPi but second on the HPC. On the Raspberry Pi, all implementations saturate at 4 threads, indicating that hardware constraints restrict performance before algorithmic differences emerge. Altogether these results suggest that across the two ARM tiers we study, cache capacity and core count are the primary drivers of implementation selection, and findings from high-end platforms do not directly transfer to systems with fewer resources.

Contents

1	Introduction	1
2	Related Work	2
2.1	Concurrent Hash Table Designs	2
2.2	Comparative Studies and the Hardware Gap	3
2.3	Benchmarking Methodology	4
3	Methods	5
3.1	Hardware Platforms	5
3.2	Software Environment	5
3.3	Implementations	7
3.4	Experimental Design	8
3.5	Thread Counts	9
3.6	Benchmarking Configuration	9
4	Results	10
5	Conclusions	19
6	Future Work	20

1 Introduction

The proliferation of edge-computing devices such as wearable health monitors, IoT sensors, and portable diagnostic tools has created a tier of hardware that must perform local, high-volume concurrent data processing under significant resource constraints. These devices rely on concurrent data structures to coordinate their access to shared state, and the design details of these structures have variable effects on throughput, latency, and energy consumption. Yet the dominant assumption when designing data structures is often that the tested hardware is abundant in cores, large last-level caches, and high memory bandwidth, which directly contradicts the needs of industries using edge hardware under cost and space constraints, such as bedside diagnostics, remote monitoring, and wearable health sensors.

Hash tables are among the most widely used concurrent data structures, serving as the backbone of in-memory stores, caches, and lookup-intensive pipelines. The existing designs span from coarse synchronization to cache-line-aligned fine-grained locking, as well as lock-free approaches. A landmark comparative study by Chen et al. [1] evaluated five concurrent hash table designs across four x86 platforms and found that no single implementation dominated across all workloads and hardware configurations. However, all five designs were tested only on high-end x86 systems with 16-244 threads and 8-320 MB last-level caches. The question of how these designs behave when the hardware budget shrinks remains unanswered.

This paper addresses this gap by extending the benchmarking study of Sestoft [2], which evaluated the same eight implementations across Apple M3, Intel i9, and Intel Xeon platforms, by applying an equivalent suite to resource-constrained ARM hardware not previously represented in that work. We evaluate eight Java implementations of concurrent hash tables across two hardware tiers: a Raspberry Pi 5 (RPi) using an ARM Cortex-A76, aarch64, and an HPC cluster node using ARM Cortex-X925 and Cortex-A725 processors, aarch64. Using the Java Microbenchmark Harness (JMH), every thread count is tested using 144 configurations, combining three read ratios (0.8/0.5/0.2), two key ranges (1K and 1M keys), and three key distributions (uniform, Zipfian-0.5, Zipfian-0.99) for every one of the eight hash table implementations.

2 Related Work

2.1 Concurrent Hash Table Designs

The development of concurrent hash tables has been shaped by two separate concerns: correctness under concurrent access, and performance optimization dependent on hardware structure.

Early concurrent designs relied on coarse-grained locking where all access is serialized through a single mutex. This is straightforward to implement but has no concurrency benefit beyond a single thread. Fine-grained locking partitions the table into independently lockable stripes, allowing concurrent access to non-overlapping regions. Java's `ConcurrentHashMap` adopted this approach, using 16 independent lock segments in JDK 5 before being redesigned in JDK 8 to use per-bucket CAS operations and to convert individual buckets from linked lists to search trees when they grow beyond a fixed size [3]. The implementations we study, `SynchronizedMap` as the coarse-grained baseline, and a family of Striped variants, allow us to isolate the effect of different methods of locking on performance across hardware tiers in more detail.

Cache-line alignment emerged as a critical optimization when researchers observed that independent data sharing the same cache line can cause severe performance degradation. David et al. [4] demonstrated this with Cache-Line Hash Table, which sizes each bucket to exactly one 64-byte cache line and eliminates stores during read operations and achieved the best scalability of all 5 tested designs. The padded variants in our study (`StripedMapPadded`, `StripedWriteMapPadded`) apply the same technique, giving us the opportunity to analyze whether padding benefits transfer to a 2 MB cache environment.

The cost of lock acquisition can carry a significant cost under high contention. When a thread acquires a lock it must perform an atomic operation on a shared location in memory, which forces other cores to invalidate their cached copy of that location. Threads that cannot acquire the lock must either busy-wait or otherwise be suspended until it is released. As more threads compete for the same lock, this waiting can noticeably affect throughput and latency.

Lock-free designs instead replace locks with atomic compare-and-swap (CAS) operations, allowing threads to read and update shared data directly and avoid the cost of lock acquisition. However, they can cause performance

degradation under high contention as threads repeatedly retry failed operations and the cost of retrying grows with increasing number of threads. The `HashTrieMap` in our study uses a concurrent hash trie [5] build on hash array mapped tries [6]. This adds a further hardware dependency, because each operation traverses multiple levels of the trie, following a pointer to a new location in memory at each step, so that a single operation may require several sequential memory fetches. This makes `HashTrieMap` more sensitive to cache capacity than the group of striped designs, and is central to our results.

2.2 Comparative Studies and the Hardware Gap

The most directly relevant previous work is Chen et al. [1], which benchmarked five designs on four multicore platforms. Their findings motivate three specific choices in our design.

Memory bandwidth. Chen et al. found that the Cache-Line Hash Table implementation saturated the Xeon E5-2630's 51.2 GB/s bandwidth at 16 threads, causing the throughput to plateau. The RPi has approximately 17 GB/s of bandwidth, and therefore we expect this plateau to arrive much earlier, potentially minimizing the scalability differences between implementations. If so, implementations that vary significantly on x86 may appear equivalent in performance on the RPi because the hardware ceiling is hit before their differences can be observed.

Zipfian distributions. The analysis of Chen et al. showed that uniform access suffered 30–53% throughput reductions compared to Zipfian access due to a lower cache hit rate from lack of hot keys. On platforms with large last-level cache, frequently accessed keys are more readily available and the bottleneck is more so from synchronization overhead. On the RPi, the 2 MB L3 cache may be insufficient to maintain even a small set of hot keys, adding cache misses on top of synchronization costs. With a highly-skewed Zipfian distribution where a small fraction of keys receives the vast majority of requests, the degradation on RPi is likely to be more significant than Chen et al. observed on their x86 platforms.

Hardware scope. All four of the platforms used by Chen et al. were x86 non-uniform memory access (NUMA) systems with 16–244 threads. The pa-

per of Chen et al. explicitly notes that many-core systems show non-trivial discrepancies from conventional multicore. Our RPi is the opposite extreme: single-socket, 4 cores, no NUMA, and smaller cache capacity. Whether the performance hierarchy from their x86 results holds here is the central empirical question of this paper. The HPC node represents the opposite end of the ARM spectrum: a many-core AArch64 system with significantly larger last-level cache and memory bandwidth, allowing us to question whether the performance gap between ARM tiers mirrors the gap Chen et. al observed between x86 tiers.

x86 vs. ARM. There is a notable gap in research on concurrent hash table implementation using ARM architecture. ARM’s Large System Extensions (LSE, ARMv8.1) add native atomic operations that reduce the CAS cost compared to load-exclusive/store-exclusive sequences in ARMv8.0. The RPi’s Cortex-A76 implements ARMv8.2-A and has access to LSE, but the JVM’s use of these instructions depends on JIT behavior, a potential source of cross-architecture differences our benchmarks capture empirically.

2.3 Benchmarking Methodology

Our benchmark design follows the approach used by Sestoft [2] for the hashmap case study, using the same implementations and workload structure but replacing the wall-clock measurement framework with the Java Microbenchmarking Harness (JMH).

In doing benchmarking on the JVM extra consideration is necessary due to the just-in-time (JIT) compiler’s runtime optimization processes. A benchmark that measures wall-clock time over looping code is therefore at risk of measuring JIT compilation overhead in early iterations and an overly optimized version in later ones, neither of which reflect the true performance. JMH addresses these issues by providing controlled warmup iterations before beginning measurement, fork-based isolation that runs each configuration in separate JVM processes to avoid cross-contamination between benchmarks, and a **Blackhole** that consumes results so the compiler doesn’t throw away unused results.

In a concurrent benchmark, some state must be shared across threads and some must be kept contained to individual threads. The implementation being tested must be shared so as to measure threads competing on it, but other state, such as the random number generator that decides whether

each operation is a read or write, must be thread-local so threads are not competing for the generator instead of the hash table. JMH's state scoping (`Scope.Benchmark` for shared state and `Scope.Thread` for thread state) makes this explicit and concise to manage.

The use of a unified harness across all eight implementations mirrors the approach of Chen et al.'s CHTBench, extended to include Zipfian key distributions via Apache Commons Math's `ZipfDistribution`.

3 Methods

3.1 Hardware Platforms

Experiments were conducted on two hardware tiers, summarized in Table 1. The Raspberry Pi 5 represents the class of resource-constrained ARM devices for which no concurrent hash table benchmarking data currently exists. The HPC node represents a high-end ARM workstation-class platform with substantially larger caches and memory bandwidth than the RPi. A cooling case was used on the RPi during benchmarking to prevent thermal throttling. The DGX Spark's unified memory is shared coherently between CPU and GPU via NVLink-C2C, though only the CPU side is exercised in this study.

3.2 Software Environment

All benchmarks were run on Java 21 using the Java Microbenchmark Harness (JMH) version 1.37. The reproducible build environment was managed via the Nix package manager using a shared Nix-flake to ensure an immutable and reproducible software environment. This ensured identical dependencies and JVM flags across both platforms. The packages imported into the system by Gradle were downloaded and transferred manually across the two testing environments to ensure that those packages were also exactly the same. Figure generation and result analysis were performed using a Python script developed with assistance from Claude (Anthropic). Prompts included requests to generate visualization code for slope charts, heatmaps, and bar charts from JMH CSV output, as well as review of the benchmarking configuration. The generated code was reviewed, tested against the benchmark data, and modified by the authors. Claude was also used to review figure cap-

	Raspberry Pi 5	HPC node (NVIDIA DGX Spark [7])
Architecture	ARMv8.2-A (AArch64)	ARMv9.2-A (AArch64)
SoC	Broadcom BCM2712	NVIDIA GB10
Cores	4× Cortex-A76 @ 2.4 GHz	10× Cortex-X925 @ 4.0 GHz + 10× Cortex-A725 @ 2.86 GHz
Topology	Single cluster	Heterogeneous DynamIQ, dual cluster
L2 cache	512 KiB private per core (2 MiB aggregate)	2 MiB private per X925 core, 512 KiB private per A725 core (25 MiB aggregate)
L3 cache	2 MiB shared	X925 cluster shares 16 MiB, A725 cluster shares 8 MiB (24 MiB aggregate)
Memory	8 GB LPDDR4X	128 GB LPDDR5X
Memory bandwidth	≈17 GB/s	≈273 GB/s
NUMA nodes	1	1
Operating system	Raspberry Pi OS (64-bit)	DGX OS 7.2.3

Table 1: Hardware platform specifications used for benchmarking.

tions and results prove for consistency with the data. All scientific analysis and interpretation of results are the authors' own.

3.3 Implementations

We evaluate eight concurrent hash table implementations, all implementing a common `OurMap<K, V>` interface with `get`, `put`, and `remove` operations. The implementations involve three design families.

Coarse-grained locking.

- **SynchronizedMap**: a hash table where everything is synchronized the map's single intrinsic lock. It is without concurrency and serves as the baseline. Any implementation that underperforms this at low thread counts has a correctness or overhead problem.

Fine-grained striped locking. All variants of the striped family partition the key space across 32 independent locks. The lock count is fixed across all runs and platforms. Its interaction with thread count is particularly interesting on the RPi, where the 4 cores mean that threads begin competing for them at a thread count well below the maximum of 32 locks.

- **StripedMap**: 32 reentrant locks, each guarding a subset of buckets. Reads and writes both acquire the relevant stripe lock.
- **StripedMapPadded**: identical to **StripedMap** but with cache-line padding inserted between lock objects to eliminate false sharing. Directly tests whether the padding benefit observed in CLHT [4] transfers to this Java implementation.
- **StripedWriteMap**: separates read and write paths, by having the data located in a "bucket array" in which the entries are the heads of linked-lists of immutable `ItemNodes`. Reads are unsynchronized (non-blocking), `put/remove` acquires a lock local to its assigned stripe of buckets, and will duplicate all of the preceding `ItemNodes` in the linked list since they are immutable. Tests for read-heavy performance gains.
- **StripedWriteMapPadded**: identical to **StripedWriteMap** but with cache-line padding inserted between lock objects to eliminate false sharing, in which two independent locks share a cache line, causing a write

to either lock to invalidate the entire line on all cores, even those accessing the other lock.

- **StripedLevelWriteMap**: uses lock striping via a two-level array structure. Rather than a single bucket array, the map maintains a top-level array of references to per-stripe sub-bucket arrays, each handled via their own locks. Reads are non-blocking and the two-level nature of it allows per-stripe resizing without affecting other stripes. Altogether prevents false cache sharing without adding padding.

Lock-free.

- **HashTrieMap**: a lock-free concurrent hash trie (CTrie) structure supporting lock-free reads and compare-and-swap based updates. Based on hash array mapped tries [6] and the concurrent trie design of Prokopec et al. [5]. Hash values are partitioned into fixed-width bit blocks, with each block used as an index at successive tree levels. CAS-based updating enables lock-free reads and writes. The compressed storage reduces memory overhead.
- **WrapConcurrentHashMap**: a wrapper around Java's built-in `ConcurrentHashMap` to match the `OurMap` interface. Serves as the practical industry baseline.

3.4 Experimental Design

Each benchmark trial runs a mixed read/write workload using these dimensions.

Read ratio. Each benchmark trial uses one of three read ratios: 0.2, 0.5, 0.8. This value is the probability that any given operation is a read. The remaining operations are writes, split evenly between `put` and `remove`. A ratio of 0.8 represents a read-heavy workload, whereas a ratio of 0.2 represents a write-heavy workload with more synchronization required.

Key range. 1,000 keys ("small") and 1,000,000 keys ("large"). The small key range is designed to stress contention around both buckets and locks. At high thread counts, threads are likely to collide on the same buckets and

produce synchronization overhead and memory access interference. The large key range tests behavior when the working set exceeds L1/L2 cache capacity and, namely on the RPi, also the L3. Before each trial, about half of the key range is pre-populated using a fixed random seed, ensuring that read operations find existing entries from the start rather than returning null.

Key distribution. Keys are drawn from one of three distributions: uniform, Zipfian with a 0.5 skew, and Zipfian with a 0.99 skew. Zipfian distributions are generated using Apache Commons Math's `ZipfDistribution` with a fixed seed for reproducibility. Keys are pre-sampled into a fixed-length array before benchmarking begins to ensure that the cost of sampling is excluded from the measurements. The combination of Zipfian-0.99 with 1,000 keys is the most extreme configuration, with a small number of keys receiving nearly all the traffic whilst the key range is small enough to guarantee high contention at higher thread counts.

This gives $3 \times 2 \times 3 = 18$ configurations per implementation, and $18 \times 8 = 144$ configurations per thread count, run across all thread counts on each platform.

3.5 Thread Counts

Thread counts were chosen based on the hardware resources of each platform:

- Raspberry Pi 5: 1, 2, 4, 8 (4 physical cores; 8 threads is included to observe behavior past the 4 core ceiling)
- HPC node: 1, 2, 4, 8, 16, 32, 64

Thread count is parameterized via JMH's `@Param` annotation. Each thread count is run as a separate JMH benchmark configuration, with all threads starting simultaneously and operating on a shared map instance.

3.6 Benchmarking Configuration

JMH was configured as follows for all runs:

- **Mode:** throughput (ops/s)
- **Warmup:** 5 iterations $\times$ 2 seconds

- **Measurement:** 10 iterations $\times$ 1 second
- **Forks:** 2 (each configuration is run in 2 separate JVM processes; results are averaged across forks)
- **Total measurement time per configuration:** 20 seconds ($2 \text{ forks} \times 10 \times 1 \text{ s}$)

Warmup iterations allow the JIT compiler to finish optimizing the benchmark code before measurements are taken, so that the recorded results reflect true performance rather than initialization. Running each configuration twice in separate JVM processes and averaging the results ensures that measurements are not skewed by the startup state or optimization choices of any single JVM instance.

Results were collected as JMH CSV outputs, one file per thread count per platform, stored in a session directory named by hostname and timestamp. Throughout this study, throughput values are aggregated using the geometric mean instead of the arithmetic mean. Performance measurements span two orders of magnitude across implementations and platforms, and the arithmetic mean is sensitive to large outliers in a way that can obscure the summary. The geometric mean, instead, captures the relative difference between measurements.

4 Results

The relative ordering of implementation performance is generally preserved between the RPi and the HPC node. `WrapConcurrentHashMap` performs best on both platforms across all access distributions, and `SynchronizedMap` performs worst on both. This confirms that the performance hierarchy established on x86 platforms transfers to ARM hardware. Hardware tier primarily affects the degree of separation between implementations, not the ordering. The interpretations in this section are inferred from throughput data alone. Direct verification using hardware performance counters and garbage collection logs is left to future work.

The most substantial rank change is from `HashTrie`, which sits seventh on the RPi but rises to second on the HPC node. `HashTrie`'s jump pushes several striped variants down the rank. `WriteMapPad` drops from second on the RPi to fourth on the HPC, `WriteMap` from fourth to sixth, and `Striped` from fifth

Implementation rank by platform — Zipfian-0.99, peak thread count
Rank 1 = highest throughput. A crossing line indicates rank changed across hardware.

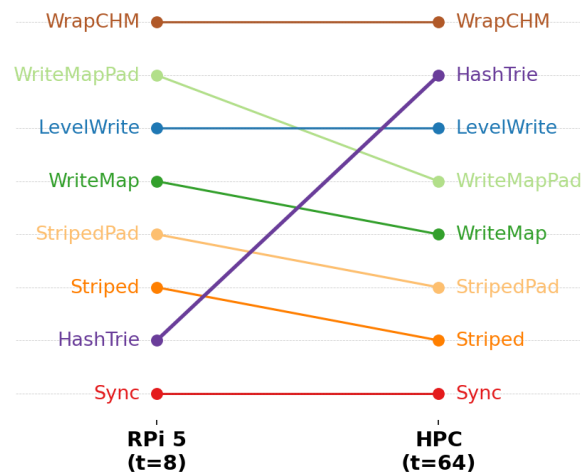


Figure 1: Implementation rank by platform under Zipfian-0.99 access at peak thread count (8 threads on RPi 5, 64 on HPC). Rank 1 denotes highest throughput. A line that crosses others indicates that implementation's rank changed across hardware tiers.

to seventh. Their absolute throughput is still higher on the HPC than on the RPi. They only lose ground relative to HashTrie. This swap between HashTrie and the striped class is the central ranking difference between the two platforms.

`SynchronizedMap`'s behavior confirms the expected cost of coarse-grained locking. Throughput drops from ~ 9 at $t=1$ to ~ 3 at $t=8$ and never recovers because each thread contributes only contention and overhead under a single global lock with no parallelism benefit.

Figure 2 shows the interaction between key range and hardware influences throughput scalability. With 1K keys on the RPi, the implementations perform similarly and flatten at 4 threads. The key set fits in the L3 cache, so we consider the primary bottleneck is the 4 core count.

On the HPC with 1K keys, the same implementations diverge substantially with thread count. `WrapConcurrentHashMap` and `HashTrieMap` continue to scale while the striped implementations saturate around 8 to 16 threads. With only 1K keys, writes frequently collide on the same buckets and cache lines and the write path bottleneck is reached well before the 32 lock or 20 core ceiling, a characteristic not shared by the `WrapConcurrentHashMap`, which allows it to dynamically scale the lock pool according to the size of the hash table. `HashTrieMap` in particular scales competitively with `WrapConcurrentHashMap` in this configuration, where frequently-accessed trie nodes remain active in cache.

With 1M keys the results degrade for the RPi across all implementations. This is because the key set far exceeds the 2 MB L3 cache, making cache misses the bottleneck regardless of implementation design. On the HPC, the 1M key set also exceeds the 24 MiB shared L3, but the HPC's much higher memory bandwidth (273 GB/s vs 17 GB/s) absorbs the cache misses. `WrapConcurrentHashMap` is the only implementation that continues scaling to 64 threads, and it does so with the largest gap above the other implementations in the 1M case. `HashTrieMap` shows the opposite trend: it is less competitive with 1M keys than with 1K keys even on the HPC. A contributing factor to this could be that the larger working set increases the likelihood of cache misses during trie traversal.

Figure 3 shows the HPC's throughput advantage over the RPi for each implementation at shared thread counts. For most implementations the advantage is roughly constant at about 2x regardless of thread count. HashTrie is the notable exception: its hardware gap widens from 1.7x at 4 threads to 2.5x at 8 threads, moving in the opposite direction to every other implemen-

Thread-count scaling — geometric mean throughput (ops/ μ s)
Top row: 1K keys (fits in RPi L3). Bottom row: 1M keys (spills out of RPi L3).
A larger drop from top to bottom on the RPi than on the HPC indicates the cache size is the bottleneck.

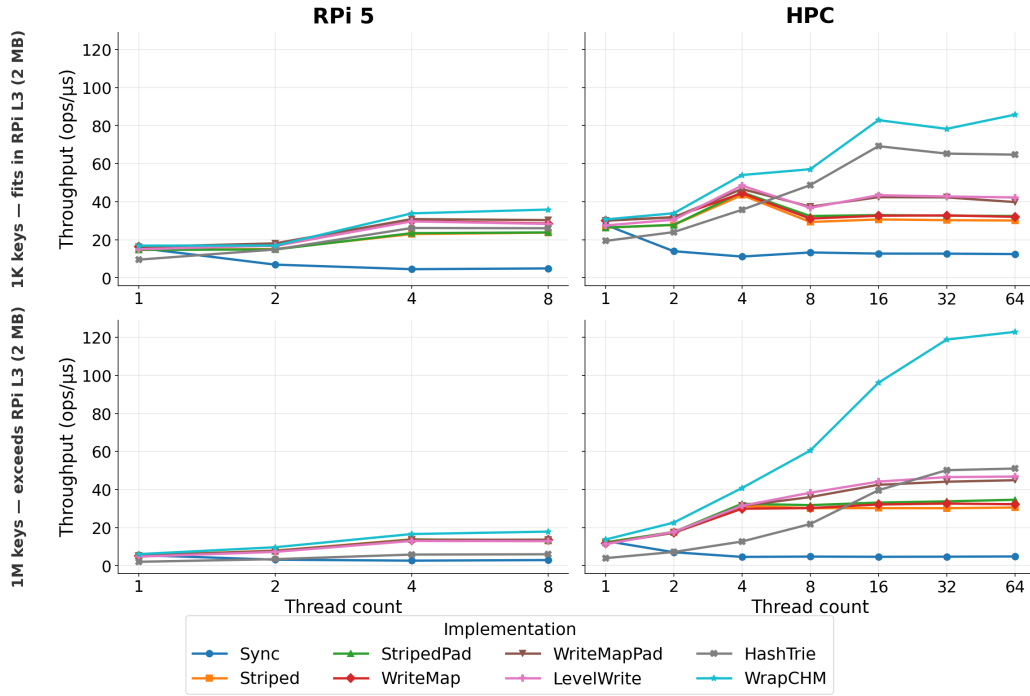


Figure 2: Geometric mean throughput (ops/ μ s) as a function of thread count, split by key range. Top row: 1K keys, which fit within the RPi 5's 2 MiB L3 cache. Bottom row: 1M keys, which exceed it. The y-axis is shared across all four panels. A larger throughput drop from top to bottom on the RPi than on the HPC indicates that cache capacity, rather than synchronisation design, is the binding constraint for large working sets on the RPi.

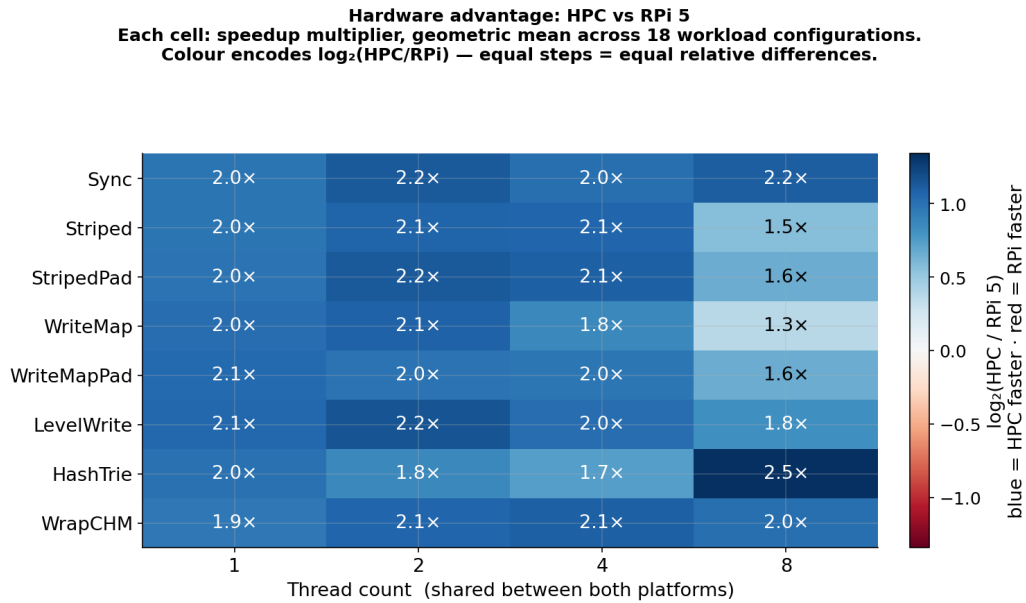


Figure 3: HPC throughput advantage over the RPi 5, expressed as a speedup multiplier at shared thread counts (1, 2, 4, 8). Each cell shows the geometric mean of the HPC/RPi speedup ratio across the 18 workload configurations for that implementation and thread count, covering three read ratios, two key ranges, and three access distributions. Color encodes the $\log_2$ of the speedup ratio so that halving and doubling appear as equal steps in opposite directions. Blue indicates the HPC is faster, red indicates the RPi is faster.

tation except Sync, which instead narrow toward or below 2x by 8 threads (for example, Striped from 2.1x to 1.5x and WriteMap from 1.8x to 1.3x). This is likely due to the pointer-chasing behavior of the implementations, in which each operation requires traveling through several nodes before reaching the target, each traversal accessing distinct locations in memory. On the RPi, with its 2 MB of L3 cache, these intermediate nodes may be evicted between accesses, increasing cache misses even at low thread counts. As thread count rises on the RPi, more threads compete for the same small cache and could evict each other's trie nodes, while the HPC's larger cache can manage the set. We suspect that this is part of why the gap widens specifically at 8 threads. Striped implementations use a flat array, and thus do not exhibit the same sensitivity to cache size, so their hardware advantage at 8 threads remains closer to the baseline, while the HashTrie implementation demonstrates the RPi's hardware disadvantage. The memory bandwidth difference may compound this, as the RPi provides approximately 17 GB/s shared across 4 cores, while the HPC provides approximately 273 GB/s across 20 cores, meaning each core on the RPi has substantially less bandwidth available for the pointer-chasing traversal that `HashTrieMap` requires on every operation.

A further potential contributing factor is garbage collection (GC) pressure. `HashTrieMap`'s immutability requirement means that every write allocates new nodes and discards old ones, continuously generating garbage. On the RPi, garbage collection pauses affect all four cores simultaneously, interrupting benchmark threads across the board. On the HPC, the same pauses are distributed across 20 cores and amortised over higher overall throughput, reducing its relative impact.

Figure 4 shows that the effect of a high Zipfian skew (0.99) is not uniform across hardware tiers, nor key ranges.

With 1K keys, both platforms suffer from high skew, but the HPC is affected more strongly (roughly +37% to +63% across the striped implementations and WrapCHM, against +18% to +23% on the RPi). At a small key range, the Zipfian distribution concentrates write traffic on the same buckets and cache lines across many threads, and contention only grows as thread count increases. Despite its larger cache, the HPC hurts more than the RPi with the exception of `SynchronizedMap`. Since all operations already pass through a single global lock, skew cannot make contention any worse. Instead, it maintains a subset of keys in cache, so operations complete slightly faster.

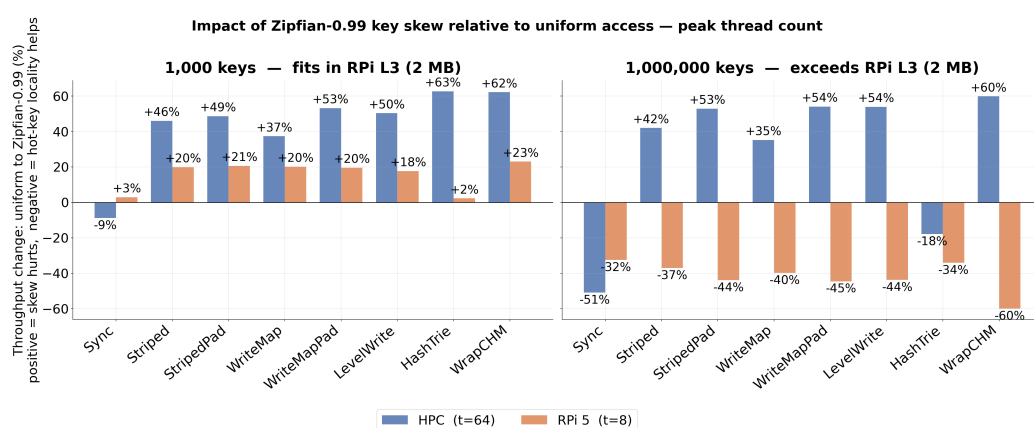


Figure 4: Throughput change from uniform to Zipfian-0.99 access at peak thread count, shown separately for the 1,000-key range (left), which fits within the RPi 5’s 2 MB L3 cache, and the 1,000,000-key range (right), which exceeds it. Positive values indicate that skewed access reduces throughput relative to uniform distribution. The graph only shows the benefit or penalty that each implementation receives due to skewed access patterns, but doesn’t reveal anything about the implementations performances relative to each other. Each bar shows the median percentage change across all read ratios (0.2, 0.5, 0.8) at peak thread count.

With 1M keys, the results vary by platform. On the RPi, skew helps across all implementations (-32 to -60%). This is because uniform access over 1M keys far exceeds the 2 MB L3 cache and leads to consistent cache misses. With an extreme skew, the work shrinks to a subset small enough to then fit into the L3 cache and reduce cache misses. The HPC node experiences the reverse: high skew reduces throughput (+35% to +60% across the striped implementations and WrapCHM). With 64 threads all targeting the same subset, contention outweighs any potential benefits from cache locality. HashTrie is the exception on both platforms, benefiting from skew at 1M keys (-21% on the HPC and -36% on the RPi). While the HashTrie implementation invalidates nodes in the cache on write operations, due to the nodes immutability, the penalty for lock contention for WrapCHM appears to outweigh the penalty of evicting and reloading invalidated nodes for the Hashtrie map. This claim is further consistent with the fact that both HashTrie map and WrapCHM perform lock-free reads and thus should not contribute to the performance gap between the two implementations.

HashTrie's sensitivity to distribution is more readily apparent on the HPC node than on the RPi. Under high Zipfian skew, HashTrie becomes a meaningful contender against WrapConcurrentHashMap on the HPC, a distinction that is not observed on the RPi. This is because a small number of hot keys receive nearly all the traffic. For striped implementations, hot keys hash to specific stripes, and those stripes' locks have a lot of contention, which only increases with thread count. HashTrie avoids this because it has no fixed lock pool, but rather CAS operations occur directly on the relevant node. This allows contention to build only on relevant nodes, as opposed to an entire stripe shared by many keys. The same mechanism does not carry over to the RPi. With only 8 threads competing against 32 available locks, the striped implementations never reach lock saturation, regardless of skew, so HashTrie has no contention advantage. Additionally, the pointer-chasing traversal required by the HashTrie implementation results in cache misses for the RPi's small L3 cache. Altogether this makes it so that the HashTrie does not differ much from the striped implementations on the RPi regardless of skew.

Figure 5 compares throughput to read ratio and peak thread count. All implementations benefit from higher read ratios to some degree, as write operations require more synchronization. The initial question is whether the WriteMap variants, which separate the read and write paths so that reads are entirely unsynchronized, create a meaningful benefit in comparison to those which require a lock for every access. On both platforms, WriteMap

Throughput by read ratio — geometric mean across distributions and key ranges, peak thread count
A steeper slope for WriteMap variants indicates the unsynchronized read path provides benefit.

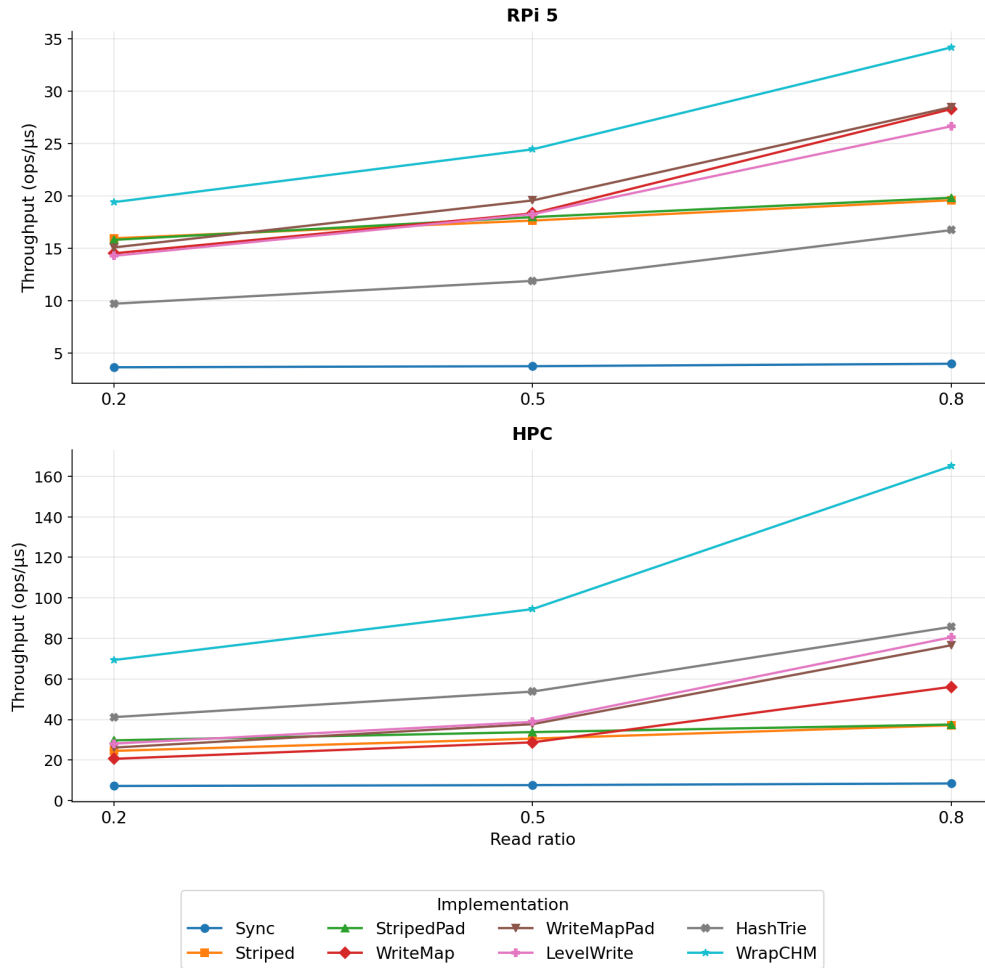


Figure 5: Geometric mean throughput by read ratio at peak thread count, aggregated across distributions and key ranges. A steeper positive slope from left to right indicates greater benefit from read-heavy workloads.

and WriteMapPad show a larger improvement from read ratio 0.2 to 0.8 than StripedMap, confirming that the unsynchronized read path provides a measurable advantage under read-heavy workloads. This effect, however, is more pronounced on the HPC, where WriteMapPad in particular shows a sharp increase and finishes substantially above StripedMap at a read ratio 0.8. This reflects the HPC node's higher thread count capacity amplifying the cost of lock acquisition on both the read path and the write path. With so many threads competing, minimizing lock contention using unsynchronized reads yields a larger relative benefit.

5 Conclusions

Three findings come out of this study. First, the performance hierarchy reported by Sestoft [2] holds on both ARM platforms: WrapConcurrentHashMap performs best and SynchronizedMap performs worst, regardless of hardware tier. Second, HashTrie showed the largest cross-tier rank change, moving from seventh on the RPi to second on the HPC. Pointer-chasing traversal performs poorly with the RPi's 2 MB L3 cache, but the absence of a fixed lock pool creates a meaningful advantage on the HPC at high thread counts and using skewed access. Third, on the RPi all implementations saturate at 4 threads, so hardware constraints create a bottleneck before algorithmic differences can even be observed. Cache capacity and core count, as opposed to implementation design, are the constraints at this tier.

Beyond these primary three, we observed the following: (1) WrapConcurrentHashMap and HashTrie are the only implementations that continue scaling past 32 threads on the HPC. The striped implementations saturate earlier due to their fixed lock count, creating an algorithmic bottleneck before the hardware ceiling is reached. (2) Cache-line padding provides a larger benefit on the HPC than the RPi. With fewer threads, false sharing is less common on the RPi, and thus the benefit of reducing it is minimal. This is consistent with Sestoft's finding that padding benefits vary with core count [2]. (3) The unsynchronized read path in the WriteMap implementations produces throughput gain under read-heavy workloads on both platforms, but the benefit is larger on the HPC where the cost of acquiring a read lock is higher when there are more active threads. (4) Skewed accesses vary with both key range and platform. With a small key set, skew increases write contention and reduces throughput on both platforms. With a large key set,

skew assists the RPi by shrinking the active set of keys to fit inside its L3 cache, but continues to hurt the HPC where 64 threads compete for a smaller subset of keys.

Altogether, based on our findings, `WrapConcurrentHashMap` is recommended across both tiers. On high-end ARM, `HashTrie` becomes competitive for read-heavy or skewed workloads with small or moderate key ranges. Padded implementations offer meaningful benefits, but only when thread counts are high enough for false sharing to matter.

6 Future Work

There are several directions that can be expanded on from the findings in this study.

Systematic review of prior work. The related work in this study focuses mostly on comparative studies. A more systematic literature review of the concurrent hash table space, covering a wider range of implementations and benchmarking methodologies would help to contextualize our findings in the broader state of the art and identify implementations and other considerations not represented yet.

Latency distribution. The current benchmarks report throughput as a single aggregate per configuration, but latency in concurrent hash tables is rarely uniform across operations. With high contention, CAS-based implementations like `HashTrieMap` can retry many times on a single operation, the results of which are concealed when using an average. Reporting latency as a histogram or percentile distribution instead of a mean would expose the variation and display each implementation's performance in more detail.

Hardware coverage. The two platforms studied here vary greatly but, nonetheless, the RPi is not, by most metrics, a truly resource-constrained computer. A more limited device, such as the BeagleBone Black with only L1 and L2 caches available, would better mimic true resource-constrained hardware and allow for more close examination of the threshold at which cache capacity transition from a binding constraint to an adequate resource.

Hardware-level profiling. The current benchmarks measure throughput as an aggregate, but do not directly observe the hardware events that drive it. Re-running experiments with hardware performance counters to measure cache miss rates, CAS retry counts, and memory bandwidth utilization per implementation would allow the cache-sensitivity arguments made for the HashTrie implementation to be verified empirically rather than inferred from broader patterns.

Hardware-adaptive implementation. The findings of this study could inform the design of a concurrent hash table that selects its concurrency strategy based on the hardware its running on. On a device with few cores and a small cache, a simple striped implementation performs as well as any other, so the overhead of a more complex design is unnecessary, if not detrimental. On a many-core system with a large cache, lock-free or WrapConcurrentHashMap-style designs make better use of the available resources. A future implementation could detect these hardware qualities at startup and select the appropriate strategy automatically.

Additional data structures. The eight implementations studied share a common interface and were selected to isolate the effects of their distinctive design choices. Further studies could explore whether the performance gains from `StripedWriteMap` come from avoiding locks on read operations, or partly from the cost of duplicating nodes on every write. A CAS-based design that allowed unsynchronized reads without copying nodes would help answer this. Furthermore, extending the study to other concurrent structures using pointers, such as lock-free skip lists, would test whether the cache sensitivity we observe with the HashTrie implementation is specifically a property of trie traversal, or a general pattern found when using pointer-chasing designs on memory-constrained hardware.

References

- [1] Z. Chen *et al.*, “Concurrent hash tables on multicore machines,” *Future Generation Computer Systems*, vol. 82, pp. 127–141, 2018.
- [2] P. Sestoft, “Multicore performance, two case studies,” Presentation, IT University of Copenhagen, 2025, updated 2025-08-18.
- [3] D. Lea, “ConcurrentHashMap source, JDK 8,” OpenJDK, 2013. [Online]. Available: <https://hg.openjdk.org/jdk8/jdk8/jdk/file/tip/src/share/classes/java/util/concurrent/ConcurrentHashMap.java>
- [4] T. David, R. Guerraoui, and V. Trigonakis, “Asynchronized concurrency: The secret to scaling concurrent search data structures,” in *ASPLOS*, 2015.
- [5] A. Prokopec, P. Bagwell, and M. Odersky, “Lock-free resizable concurrent tries,” in *LCPC*, 2011.
- [6] P. Bagwell, “Ideal hash trees,” EPFL, Tech. Rep., 2000.
- [7] NVIDIA Corporation, “NVIDIA DGX Spark User Guide: Hardware Overview,” <https://docs.nvidia.com/dgx/dgx-spark/hardware.html>, 2025, accessed: 2026-05-05.